Diagramme entité-relation de la base de données

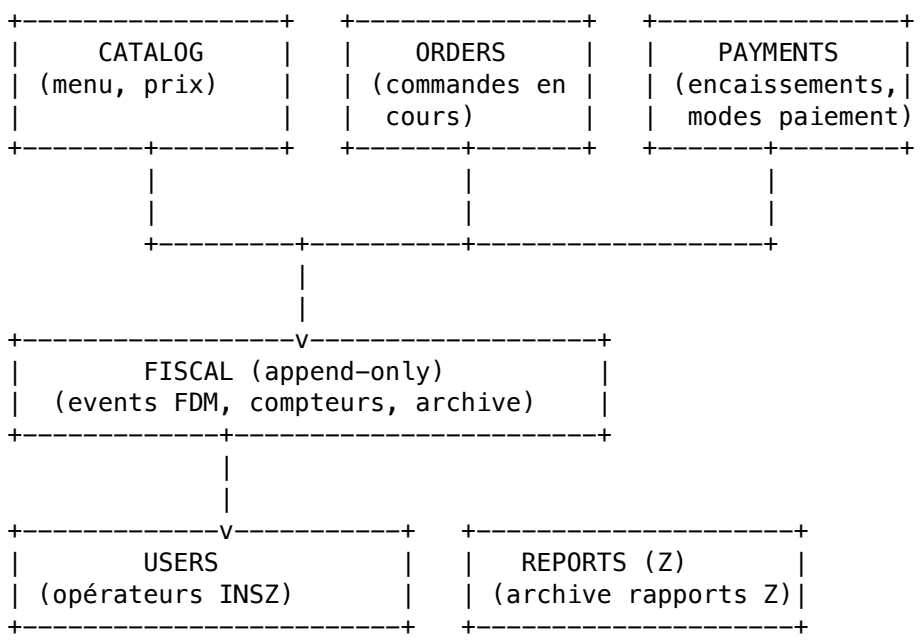
Date : 2026-05-14 **Référence :** §II.C “Relations entre les tables” du manuel de certification. **Source de vérité :** `crates/pos-db/src/schema.rs` (le SQL définit le schéma exact ; ce document reflète la révision actuelle).

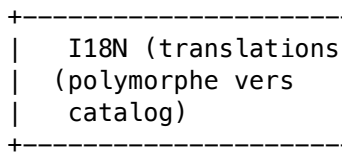
Ce document décrit visuellement les relations entre les tables de la base SQLite `pos.db`. Pour l’inventaire détaillé champ-par-champ, *voir : SCHEMA_BD_FR.md*. Pour la sécurité et les contraintes, *voir : SECURITE_BD_FR.md*.

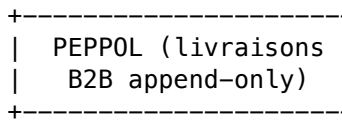
---

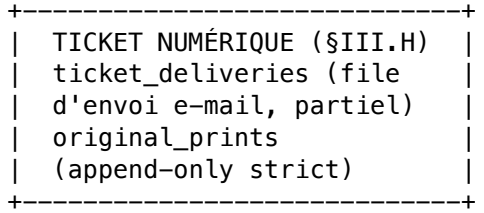
1. Vue d’ensemble par domaines

La base est organisée en 6 domaines logiques :

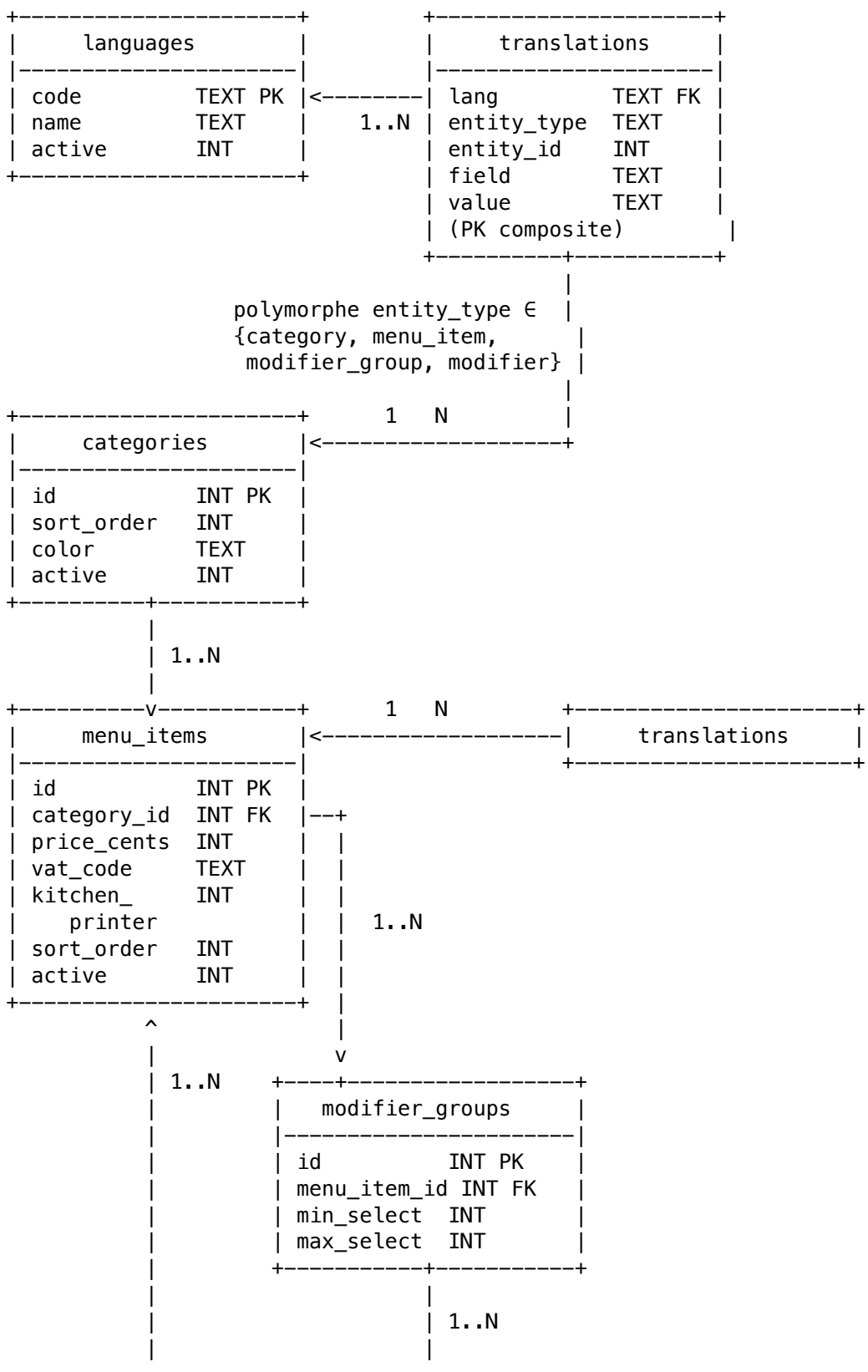


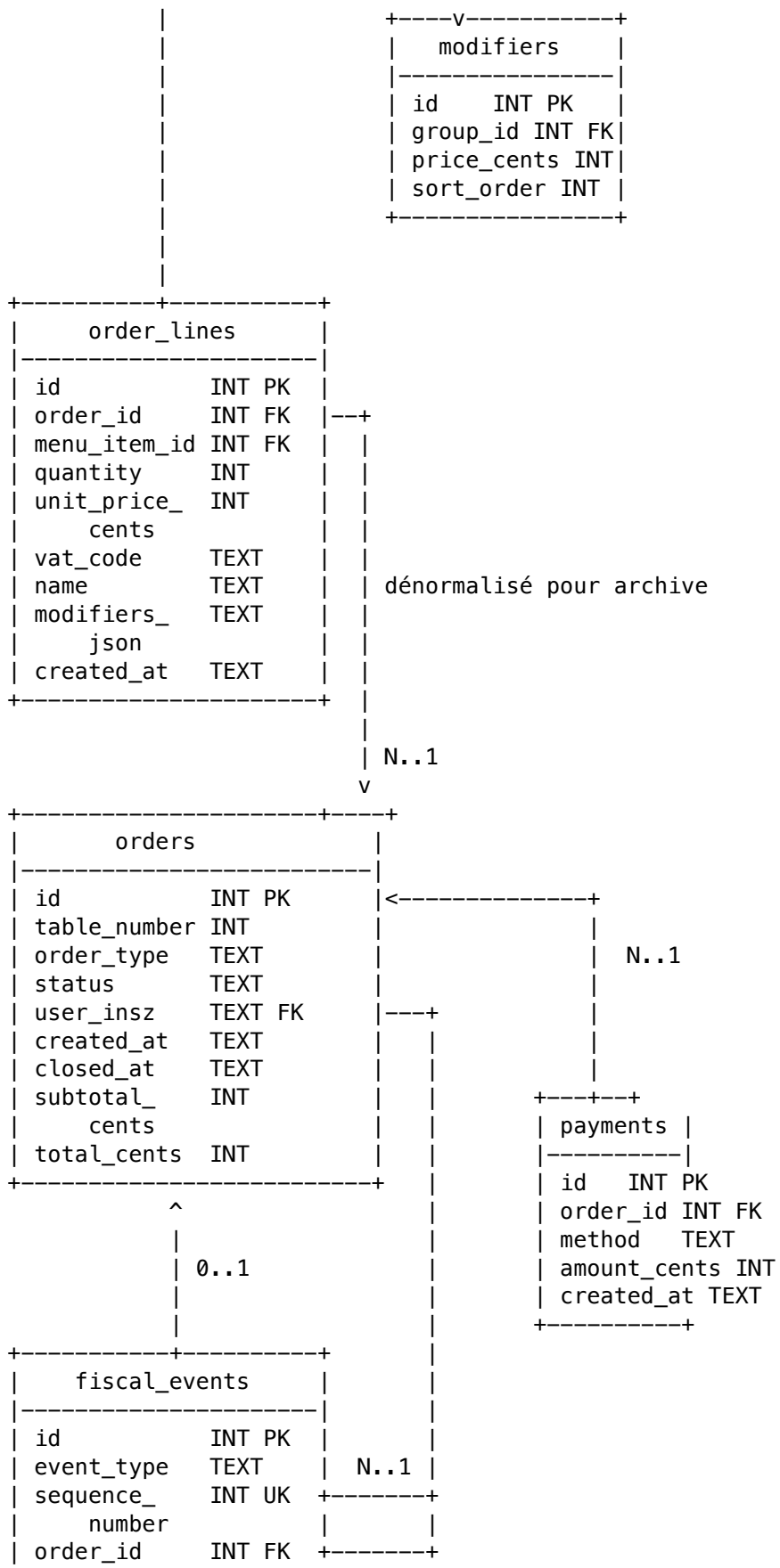


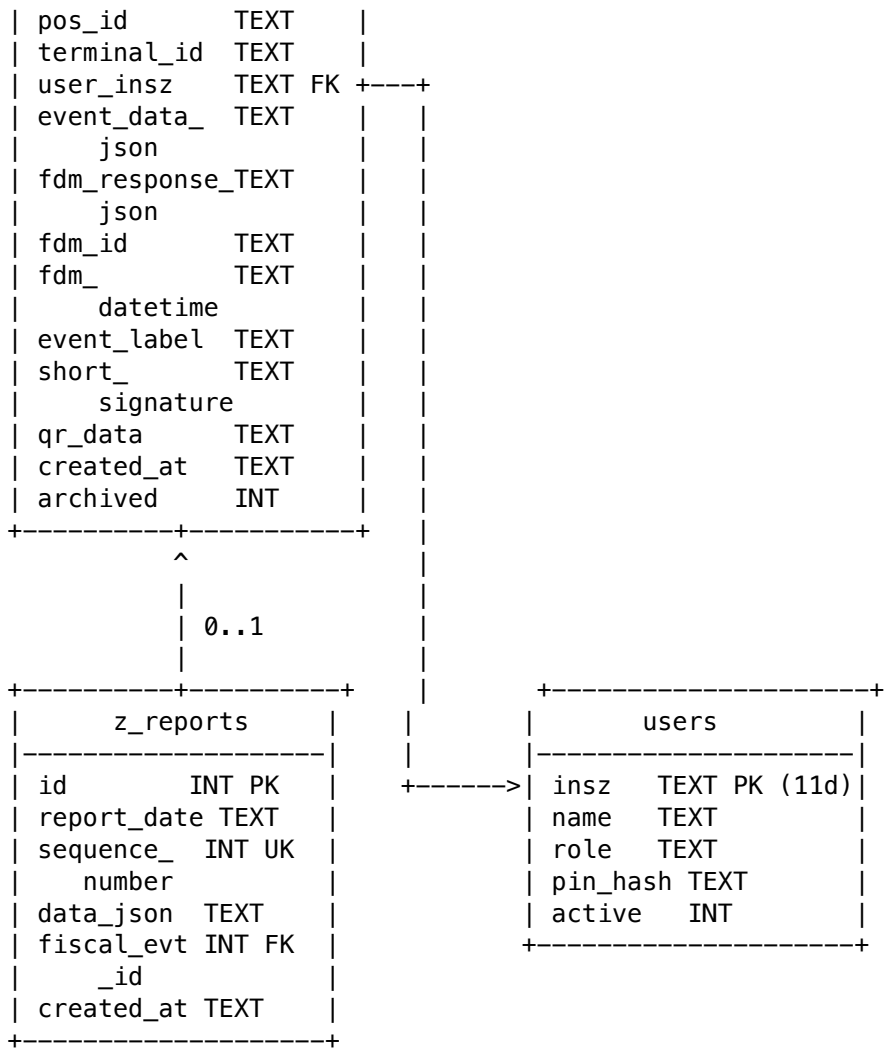


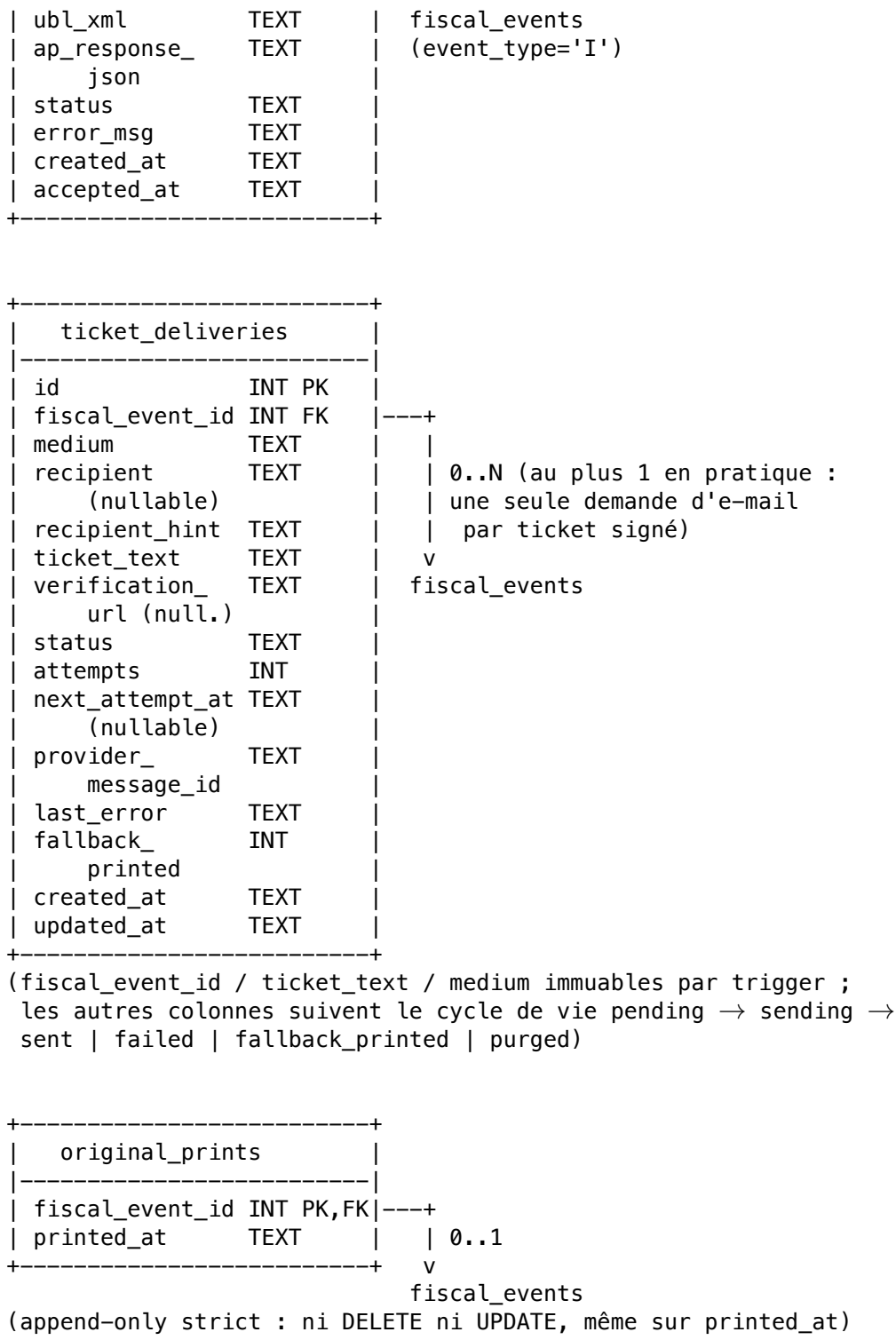


2. Diagramme ER détaillé




modifiers
id INT PK
group_id INT FK
price_cents INT
sort_order INT
order_lines
id INT PK
order_id INT FK
menu_item_id INT FK
quantity INT
unit_price_cents INT
vat_code TEXT
name TEXT
modifiers_json TEXT
created_at TEXT
dénormalisé pour archive
N..1
orders
id INT PK
table_number INT
order_type TEXT
status TEXT
user_insz TEXT FK
created_at TEXT
closed_at TEXT
subtotal_cents INT
total_cents INT
N..1
payments
id INT PK
order_id INT FK
method TEXT
amount_cents INT
created_at TEXT
0..1
fiscal_events
id INT PK
event_type TEXT
sequence_number INT UK
order_id INT FK
N..1


| pos_id TEXT |
| terminal_id TEXT |
| user_insz TEXT FK +---+
event_data_ TEXT	
json	
fdm_response_TEXT	
json	
fdm_id TEXT	
fdm_ TEXT	
datetime	
event_label TEXT	
short_ TEXT	
signature	
qr_data TEXT	
created_at TEXT	
archived INT	
+----------+----------+
^
0..1
+----------+----------+	+--------------------+
z_reports			users
--------------------|	|--------------------
id INT PK		+------>	insz TEXT PK (11d)
report_date TEXT		name TEXT
sequence_ INT UK		role TEXT
number		pin_hash TEXT
data_json TEXT		active INT
fiscal_evt INT FK	+--------------------+
_id
created_at TEXT
+--------------------+
+--------------------+
fiscal_counters
event_label TEXT PK
current_value INT
+--------------------+
(table autonome, mise à jour atomique dans la même transaction
SQLite qui ajoute à fiscal_events, sous le verrou de signature)
+--------------------------+
peppol_deliveries
id INT PK
fiscal_event_id INT FK |---+
invoice_no TEXT	
customer_ TEXT	
endpoint_id	
ap_vendor TEXT		1..1
ap_message_id TEXT	v



---

3. Relations principales (verbalisées)

De → Vers	Cardinalité	Sémantique
categories → menu_items	1 → N	Une catégorie contient N articles de menu
menu_items → modifier_groups	1 → N	Un article peut avoir N groupes de modificateurs (sauce, cuisson, …)
modifier_groups → modifiers	1 → N	Chaque groupe contient N options (Soja, Wasabi, …)
orders → order_lines	1 → N	Une commande contient N lignes
order_lines → menu_items	N → 1	Référence dénormalisée vers l'article (nom + prix copiés)
orders → payments	1 → N	Une commande peut être réglée par N paiements (split)
orders → fiscal_events	1 → 0..N	Une commande peut générer 0 (open) à plusieurs events fiscaux (P → N, ou N + C copie, etc.)
fiscal_events → orders	N → 0..1	Un event fiscal référence 0 (events système comme S, F-WAR) ou 1 commande
users → orders	1 → N	Un opérateur ouvre N commandes
users → fiscal_events	1 → N	Un opérateur émet N events (via user_insz)
z_reports → fiscal_events	1 → 1	Chaque Z archive un event R correspondant
peppol_deliveries → fiscal_events	1 → 1	Chaque livraison Peppol est liée à un event I
ticket_deliveries → fiscal_events	0..N → 1	Une ligne par ticket dont le support demandé inclut l'e-mail (DIGITAL/PAPER_DIGITAL) ; ajoutée dans la même transaction que l'event N/T/C correspondant
original_prints → fiscal_events	0..1 → 1	Au plus une ligne par événement dont l'original a été imprimé (contrainte de clé primaire = garde applicative contre une seconde impression)
translations → (polymorphe)	N → 1	Traduction d'un champ d'une entité (category / menu_item / modifier_group / modifier) dans une langue (FR / NL / EN / ZH)
translations → languages	N → 1	Référence le code langue
fiscal_counters (autonome)	-	Pas de FK ; clé event_label □ {N, P, T, C, I, F, S, R}

---

4. Append-only et intégrité

Ces tables sont **append-only** au niveau applicatif (jamais de DELETE, et pas d'UPDATE hors exceptions ciblées) :

- fiscal_events - archive des events fiscaux signés par le FDM
- z_reports - archive des rapports de clôture
- peppol_deliveries - archive des livraisons B2B Peppol
- original_prints - un original n'est jamais réimprimé silencieusement (§8.1) ; ni DELETE ni UPDATE, aucune exception

- `ticket_deliveries` - append-only **partiel** : `fiscal_event_id`, `ticket_text` et `medium` sont immuables par trigger, mais les autres colonnes (`status`, `attempts`, `recipient`…) évoluent volontairement pour suivre le cycle de vie de l'envoi (file d'attente, pas une archive figée)

Pour les autres tables (catalogue, commandes en cours, paiements, utilisateurs) les opérations UPDATE/DELETE sont autorisées via la couche commandes (avec audit trail dans les logs systemd-journald). Les `orders` sont marquées `status='closed'` ou `status='voided'` plutôt que supprimées.

La contrainte `fiscal_events.sequence_number UNIQUE` empêche tout doublon de compteur fiscal au niveau base. La table `fiscal_counters` est mise à jour atomiquement dans la même transaction SQLite que l'insert dans `fiscal_events`, sous le verrou de signature unique (cf. *STRATEGIE_COMPTEURS_FR.md* §5). Les requêtes de signature en attente de réponse sont conservées dans `pending_signs` (cf. *SCHEMA_BD_FR.md* §3.9).

---

5. Indexes

Table	Index	Colonnes	Raison
`translations`	`idx_translations_lookup`	`entity_type, entity_id, lang`	Lookup rapide pour le rendu UI multilingue
`fiscal_events`	(implicite via PK + UNIQUE)	`id`, `sequence_number`	Lookup par ID et garantie d'unicité du compteur
`orders`	(implicite via PK)	`id`	Lookup par ID
`users`	(implicite via PK)	`insz`	Auth login
`ticket_deliveries`	`idx_ticket_deliveries_due`	`status, next_attempt_at`	Sélection rapide des envois dus par le worker de réessai (§III.H)

Des indexes additionnels peuvent être ajoutés en post-cert si des problèmes de performance apparaissent à plus grande échelle (multi-restaurant). Pour le périmètre mono-terminal (~150 tickets/jour pour un restaurant type), aucun index supplémentaire n'est nécessaire.

---

6. Tables des commandes en ligne

Les tables des commandes en ligne (`webshop_orders`, `hubrise_channels`, `hubrise_outbox`, `order_price_changes`, `integration_switches`, `integration_switch_log`, migrations 20 à 25) sont décrites dans `SCHEMA_BD_FR.md` §5. Elles ne sont écrites que si l'intégration est activée pour l'installation (`COMMANDES_EN_LIGNE_FR.md` §2), sauf le journal de l'interrupteur, qui reçoit au moins la valeur de départ. `webshop_orders.order_id` référence `orders.id` ; ses colonnes `p_event_id`, `n_event_id` et `refund_event_id` référencent `fiscal_events.id`.

---

7. Versionnement du schéma

Le schéma est versionné via `pos-db::migrate::run_migrations`. La table `_migrations_applied` enregistre chaque migration appliquée (`version`, `name`, `applied_at`). Aucune modification de schéma post-cert n'est possible sans une nouvelle demande de certification (Article 109 du manuel).

Version actuelle du schéma : v7 (`initial-schema`, `catalogue-fiscal-fields`, `booking-periods`, `fiscal-archive-append-only`, `ticket-deliveries`, `original-prints`, `german-language`) - liste exacte dans `SCHEMA_BD_FR.md` §6 et `crates/pos-db/src/migrate.rs::MIGRATIONS`.